Understanding Gaussian Mixture Models

Soft clustering with Expectation–Maximisation on the Palmer Penguins dataset

Tutorial focus. This tutorial explains how a Gaussian Mixture Model (GMM) represents clusters as probability distributions rather than rigid labels. The worked example uses two penguin bill measurements to show three ideas: component shape, soft membership probabilities and practical model selection with BIC.

1. Introduction

Clustering is usually described as grouping unlabeled data into clusters where the items in the same group are similar to each other and different from those in other groups. That is a helpful starting point, but it leaves out an important issue: does each data point really belong completely to just one group? In many real-world datasets, the answer is no. Some observations sit close to the boundary between groups, share features with more than one cluster, or form patterns that are stretched and uneven rather than neat and circular.

Gaussian Mixture Models are useful because they approach clustering in a more flexible way. Instead of forcing each data point into a single group, a GMM assumes the data come from several overlapping Gaussian distributions, each with its own Centre, shape and size. It then works out the probability that each observation belongs to each component. This makes GMM helpful not just for clustering, but also for tasks like density estimation, identifying unusual observations, and analyzing data where uncertainty is an important part of the result.

This tutorial aims to explain the main idea behind Gaussian Mixture Models and the Expectation–Maximization (EM) algorithm used to fit them. It then applies the method to the Palmer Penguins dataset and uses a small set of figures to show how the model works in practice, including the overall data pattern, the shape of the components, uncertain cases and the model's level of confidence.

2. What is GMM models

A Gaussian Mixture Model assumes that the data come from a combination of several Gaussian distributions. Each component has three main parts: a mixture weight, a mean, and a covariance matrix. The weight shows how much that component contributes to the overall dataset, the mean marks its Centre, and the covariance matrix describes its shape, including how spread out it is and whether it is tilted in a particular direction.

The central idea can be written as:

$$p(x) = \Sigma_k \pi_k N(x \mid \mu_k, \Sigma_k)$$

This equation means that the probability of a data point is built from the combined contribution of all the Gaussian components in the model. The important point is that the model does not force a point into just one cluster. Instead, it assigns a probability to each possible component. For example, if one penguin has probabilities of [0.93, 0.06, 0.01], the model is very confident about where it belongs. If another has [0.52, 0.44, 0.04], its membership is much less clear. That

probabilistic way of thinking is what makes GMM so useful, because it reflects uncertainty rather than pretending every observation fits neatly into a single group.

A useful way to picture this is that each Gaussian component acts like a soft cluster with its own centre, spread and orientation.

Another strength of GMM comes from the covariance matrix. Because it captures both spread and direction, the model can represent clusters that are stretched out or tilted rather than perfectly round. This is useful in real data when features change together. For example, if longer bills are often associated with shallower bill depths, a GMM component can follow that diagonal pattern instead of forcing the cluster into a circular shape.

```
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

X = penguins[["bill_length_mm", "bill_depth_mm"]].to_numpy()
X_scaled = StandardScaler().fit_transform(X)

gmm = GaussianMixture(
    n_components=3,
    covariance_type="full",
    random_state=42,
    n_init=10,
)
gmm.fit(X_scaled)
```

3. How EM learns the components

These parameters are not known beforehand, so the model has to learn them from the data. GMMs are usually fitted using the Expectation–Maximization (EM) algorithm. EM is an iterative method designed for models with hidden structure. In the case of a GMM, the hidden part is the component membership of each observation, since we do not directly know which Gaussian generated each point.

In the E-step, the model keeps the current parameter values fixed and calculates the responsibility of each component for each observation. In other words, every point is given a set of probabilities showing how likely it is to belong to each component, and those probabilities add up to one. In the M-step, the model uses those probabilities to update the mixture weights, means and covariance matrices. Components that explain more points receive more weight, the means shift towards the observations they fit best, and the covariance matrices adjust their size and orientation to match the local pattern in the data.

This process repeats until the improvement in fit becomes very small. One reason EM is so useful is that each stage is quite intuitive. The E-step asks, “given the current components, how well does each one explain this point?” The M-step asks, “given those soft assignments, how should the parameters be updated to fit the data better?” In practice, EM can settle on a local optimum rather than the absolute best solution, which is why the notebook uses multiple initializations (n_init=10).

4. Worked example: Palmer Penguins

For the worked example, I use the **Palmer Penguins dataset**, which is a small real-world dataset containing measurements from Adelie, Chinstrap and Gentoo penguins. I focus on just two

features, bill length and bill depth, because they are easy to plot in two dimensions while still showing some overlap between groups. After removing rows with missing values, the analysis includes 342 penguins. The species labels are not used when fitting the GMM; they are only included later to help interpret the structure of the data. The local CSV is loaded and reduced to the two bill features used in the plots:

```
penguins = pd.read_csv("penguins.csv").dropna()
X = penguins[["bill_length_mm", "bill_depth_mm"]].to_numpy()
species = penguins["species"]
```

Figure 1 shows the labelled data. Even before fitting the model, two things are clear. First, the species do not form perfectly separate circular groups; they vary in both spread and direction. Second, some observations lie in overlapping regions, especially where bill length is similar for Chinstrap and Gentoo penguins. These are the kinds of cases where a probabilistic clustering method can be more helpful than a strict one-group assignment.

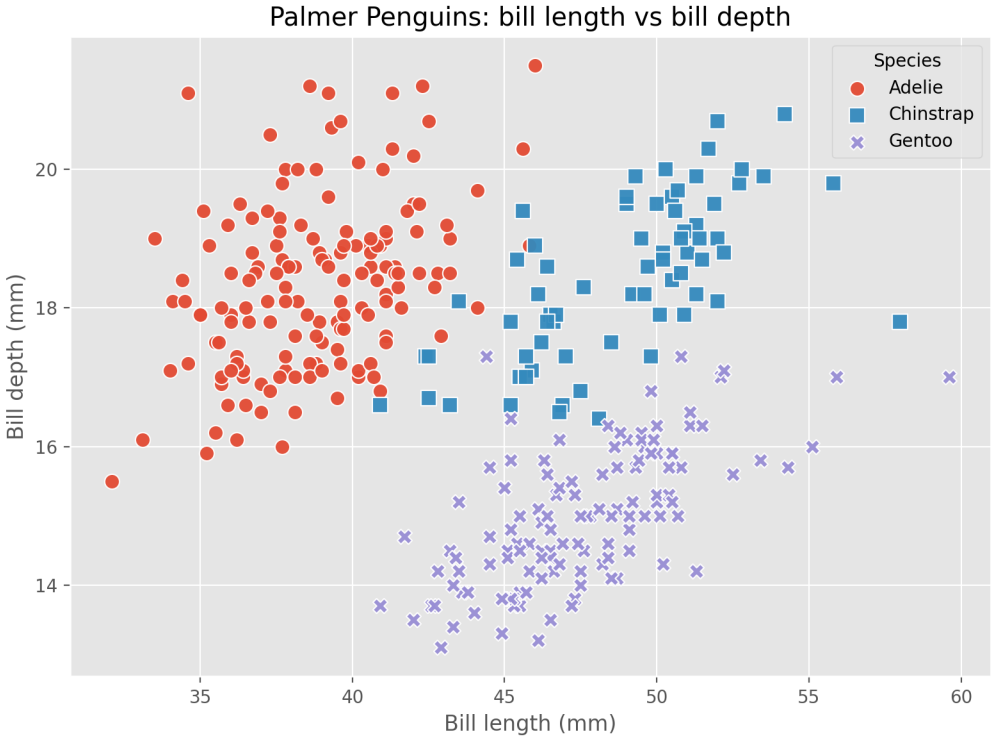

Figure 1. Species labels shown only for context. The GMM is fitted without using these labels.

5. Results and interpretation

I fit a three-component GMM with full covariance matrices. Using the full covariance setting allows each component to have its own shape and orientation, which makes sense here because the clusters are not simple round groups. Figure 2 is the main teaching figure. The ellipses show the covariance structure learned for each Gaussian component, while the points are coloured according to the component they are most likely to belong to.

Two things stand out. First, the components do not all have the same shape: one is fairly compact and lower in bill depth, another is taller and more vertical, and a third sits further to the left with a wider spread. Second, the ellipses overlap. That overlap is not a weakness of the model; it is part of what makes GMM useful. Instead of forcing every observation into a completely separate region, the model allows overlap and captures the uncertainty in where some points belong.

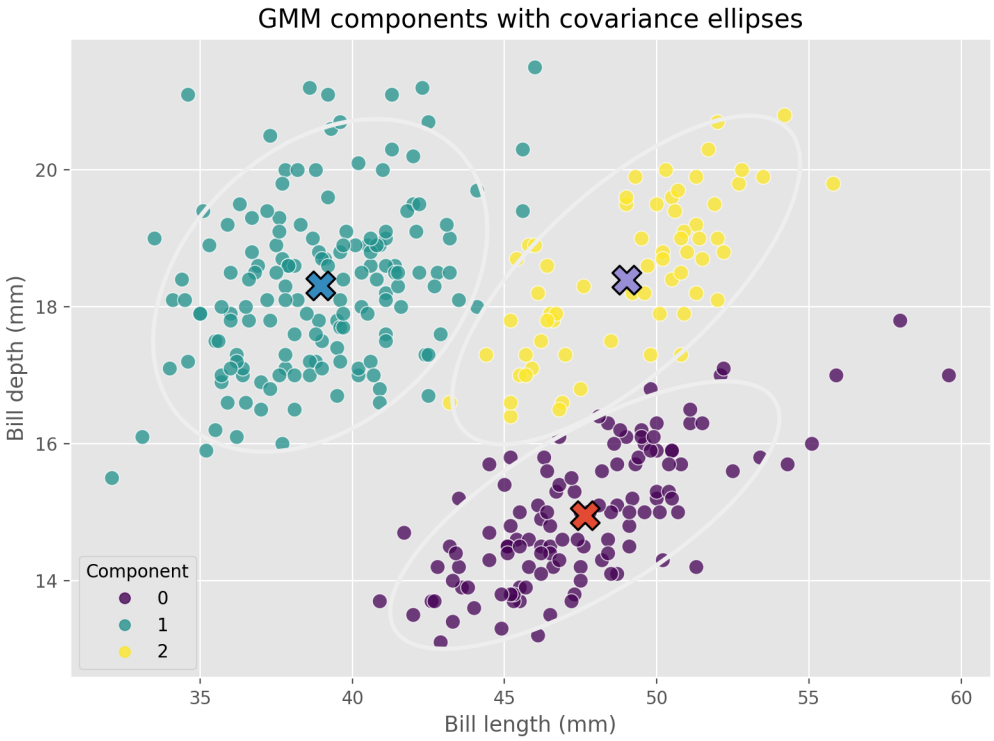

Figure 2. Learned GMM components with covariance ellipses.

Soft clustering becomes much easier to understand when we look directly at the observations the model finds most uncertain. Figure 3 shows the posterior probabilities for the ten penguins with the lowest maximum component probability. Instead of assigning each penguin to just one group, the model spreads probability across several components. In some cases one component is still clearly dominant, but in others the probabilities are shared much more evenly. This is the kind of information that a hard clustering method cannot show very well.

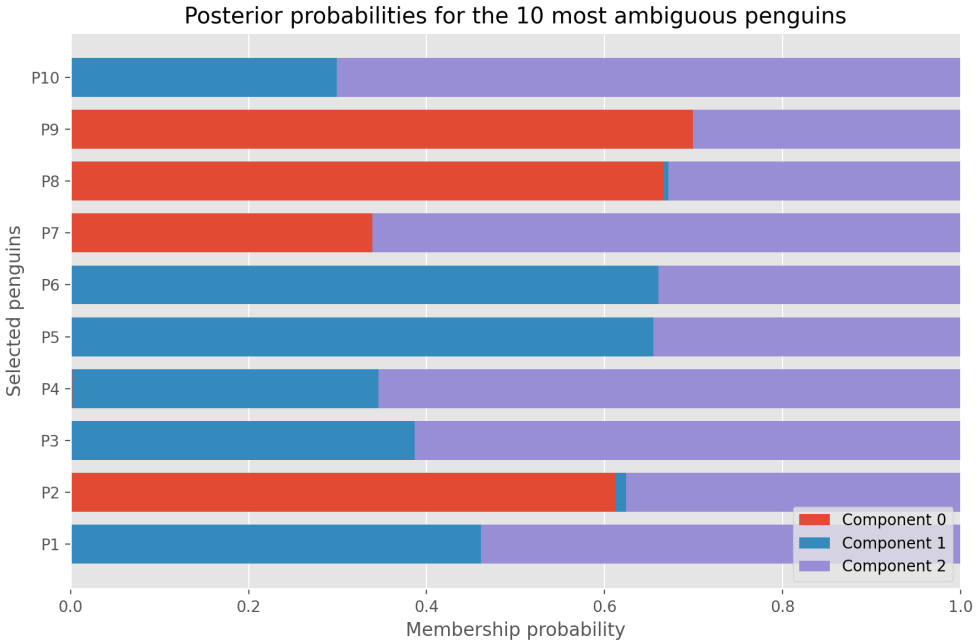

Figure 3. Posterior membership probabilities for the ten most ambiguous penguins. Each bar sums to one.

Figure 4 gives a broader view of confidence across the whole dataset. Most penguins have very high maximum posterior probabilities, which suggests that the model is usually quite certain about their membership. In the notebook, the average maximum posterior probability is 0.977, while the lowest value is 0.539. That pattern is useful because it shows that GMM is not uncertain all the time. Most observations are assigned with high confidence, and uncertainty mainly appears in sensible boundary areas.

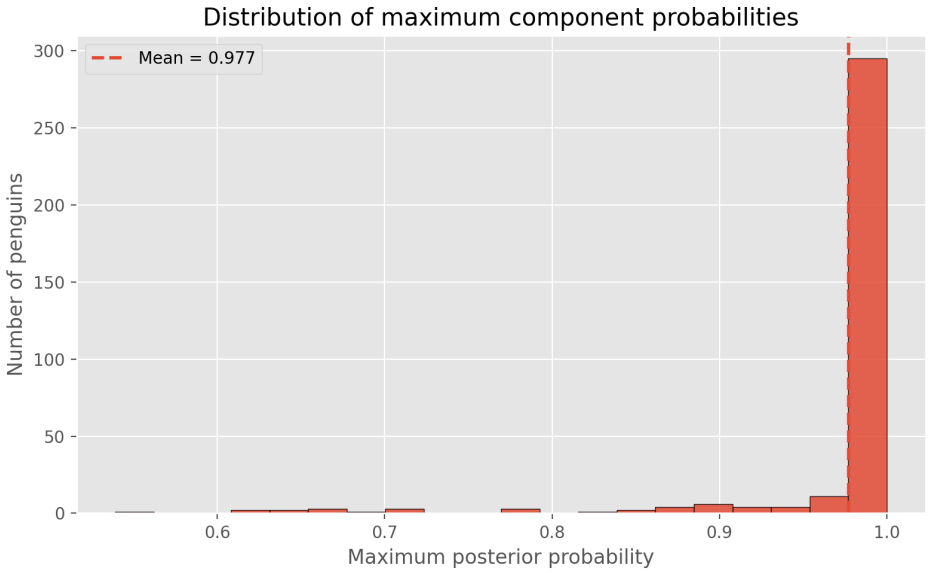

Figure 4. Distribution of the maximum posterior probability. Values near one indicate confident assignments.

```
probs = gmm.predict_proba(X_scaled)
max_prob = probs.max(axis=1)

# Lower values indicate more ambiguity
ambiguous_idx = np.argsort(max_prob)[:10]

# BIC for model selection
bic = gmm.bic(X_scaled)
```

A final practical point is model selection. When using a GMM, you need to decide both how many components to include and what kind of covariance structure to use. Figure 5 compares several possible models using the Bayesian Information Criterion (BIC). Lower BIC values suggest a better balance between how well the model fits the data and how complex it is. In this case, the lowest BIC in the search comes from a tied-covariance model with four components, even though the tutorial mainly uses a three-component model because it is easier to explain.

The BIC search is implemented as a short loop over candidate numbers of components and covariance settings:

```
records = []
for cov in ["full", "tied", "diag", "spherical"]:
    for k in range(1, 7):
        model = GaussianMixture(
            n_components=k, covariance_type=cov, random_state=42
        )
        model.fit(X_scaled)
```

```
        records.append((cov, k, model.bic(X_scaled)))
```

This is better seen as a useful detail than a contradiction. In unsupervised learning, the model that fits the data best does not always have to match known human labels exactly, and the model that is easiest to explain is not always the one with the very lowest BIC. I kept the three-component model as the main example because it is easier to interpret and still matches the visible structure of the bill measurements quite well. In the notebook, an external comparison with the species labels gives an adjusted Rand index of 0.912, which suggests that the clustering found by the model is already very close to the known species pattern, even though those labels were not used during fitting.

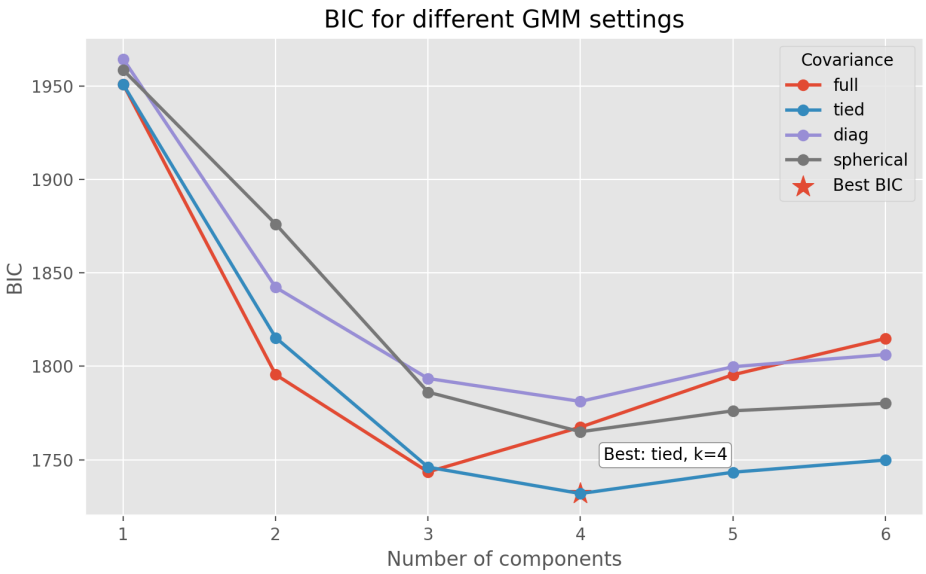

Figure 5. BIC comparison across different numbers of components and covariance settings

6. When GMM is useful

In practice, I would choose a GMM when I expect clusters to overlap, vary in spread, or contain some genuine uncertainty. It is especially useful when I want more than just a final cluster label. The posterior probabilities can help identify borderline cases, show how confident the model is, and highlight observations that do not fit neatly into a single component. Because of this, GMM can also be useful as a density model, not just as a clustering method.

At the same time, it does have limitations. The results depend on choices such as the number of components and the covariance type, and the EM algorithm can sometimes settle on a local optimum rather than the best possible solution. GMM also relies on Gaussian assumptions, so it may not work as well when cluster shapes are very irregular. BIC can help with model selection, but it should be used alongside subject knowledge and visual inspection rather than on its own.

7. Conclusion

Gaussian Mixture Models are useful because they treat clustering as a problem of probability rather than a simple yes-or-no assignment. Instead of forcing each observation into one fixed group, they estimate how strongly each component is associated with that point. The Palmer Penguins example shows why this is helpful: the learned components have different shapes, most penguins are assigned with high confidence, and a smaller number are sensibly recognised as uncertain. For students and practitioners, the main lesson is straightforward: GMM is a good choice when uncertainty and cluster shape are meaningful parts of the data rather than something to ignore.

References

1. Giesen, J., Baur, T., Schreiber, F. and others (2024) 'The whole and its parts: Visualizing Gaussian mixture models', *Visual Informatics*.
2. Chen, X. and Zhang, A. (2024) 'Achieving Optimal Clustering in Gaussian Mixture Models with Anisotropic Covariance Structures', *Advances in Neural Information Processing Systems (NeurIPS 2024)*.
3. Scrucca, L. (2024) 'A Model-Based Clustering Approach for Bounded Data Using Transformation-Based Gaussian Mixture Models', arXiv preprint.
4. Puchhammer, P., Wilms, I. and Filzmoser, P. (2025) 'Outlier-Robust Multi-Group Gaussian Mixture Modeling with Flexible Group Reassignment', arXiv preprint

GitHub Repository: https://github.com/mi24acr/Machine_Learning_Tutorial_Hasan.git